Laporan End 2 End Testing Siakad

Penjaminan Mutu Perangkat Lunak



Disusun Oleh :

Nama : Fadhil Taufiqurrachman
NIM : 244107020090
Kelas : 3G
Absen : 06

Program Studi D4 Teknik Informatika

Jurusan Teknologi Informasi

Politeknik Negeri Malang

Tahun 2026/2027

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak modern, pengujian merupakan tahapan krusial yang menjamin kualitas dan keandalan sistem sebelum digunakan oleh pengguna akhir. Salah satu jenis pengujian yang paling komprehensif adalah End-to-End (E2E) Testing, yaitu pengujian yang memvalidasi keseluruhan alur kerja aplikasi dari sudut pandang pengguna nyata.

Sistem Informasi Akademik (SIAKAD) Politeknik Negeri Malang merupakan aplikasi web yang digunakan oleh seluruh civitas akademika untuk mengelola data akademik, termasuk kartu rencana studi (KRS), nilai, jadwal perkuliahan, dan presensi mahasiswa. Sebagai sistem yang diakses oleh ribuan pengguna, SIAKAD perlu dijamin keandalan alur-alur kritisnya melalui pengujian otomatis yang sistematis.

Laporan ini mendokumentasikan implementasi pengujian E2E pada SIAKAD Polinema menggunakan framework Playwright dengan bahasa TypeScript, yang diintegrasikan ke dalam pipeline CI/CD GitHub Actions.

1.2 Tujuan

1. Merancang dan mengimplementasikan skenario pengujian E2E pada dua alur kritis SIAKAD Polinema, yaitu alur autentikasi pengguna dan alur modul presensi mahasiswa.

2. Menerapkan pola desain Page Object Model (POM) untuk memisahkan logika pengujian dari detail teknis antarmuka pengguna.

3. Menggunakan strategi pengujian yang resilien terhadap latensi jaringan dan perilaku asinkron aplikasi web legacy.

4. Mengintegrasikan pengujian ke dalam pipeline CI/CD GitHub Actions sebagai quality gate otomatis.

1.3 Ruang Lingkup

Pengujian dilakukan pada dua alur kritis berikut:

Alur 1 – Autentikasi Pengguna: Meliputi skenario login gagal (negative path) dan login berhasil (happy path).

Alur 2 – Modul Presensi Mahasiswa: Meliputi navigasi ke menu Akademik > Presensi, pemilihan tahun ajaran melalui dropdown, klik tombol Filter, serta verifikasi tabel data absensi (Alpha, Ijin, Sakit) dan baris rekapitulasi Total AIS.

BAB 2 DASAR TEORI

2.1 End-to-End (E2E) Testing

End-to-End Testing adalah metode pengujian perangkat lunak yang memvalidasi keseluruhan alur kerja aplikasi secara terintegrasi, dari antarmuka pengguna (Frontend UI) hingga basis data (Database), termasuk komunikasi jaringan, API backend, dan layanan pihak ketiga. E2E Testing mensimulasikan perilaku pengguna nyata untuk memastikan seluruh komponen sistem bekerja secara harmonis.

Berdasarkan piramida pengujian (Mike Cohn / Martin Fowler), E2E Testing berada di puncak piramida dengan proporsi sekitar 10% dari keseluruhan pengujian. E2E difokuskan pada alur bisnis kritis (critical user journeys) yang bernilai tinggi bagi pengguna dan organisasi.

2.2 Playwright

Playwright adalah framework pengujian otomatisasi web modern yang dikembangkan oleh Microsoft. Playwright mendukung pengujian lintas browser (Chromium, Firefox, WebKit) secara native dan memiliki fitur unggulan seperti:

• Auto-waiting: Playwright secara otomatis menunggu elemen siap sebelum berinteraksi, mengurangi flaky tests.

• Web-First Assertions: Asersi yang melakukan polling berkala hingga kondisi terpenuhi, menghindari kebutuhan hardcoded sleep.

• Trace Viewer: Alat debugging visual yang merekam DOM snapshots, network logs, dan screenshots per langkah pengujian.

• Multi-tab dan Multi-origin: Mendukung pengujian alur yang melibatkan banyak tab browser dan domain berbeda.

2.3 Page Object Model (POM)

Page Object Model adalah pola desain arsitektural yang memisahkan abstraksi elemen antarmuka pengguna ke dalam kelas-kelas tersendiri (Page Objects). Setiap halaman web direpresentasikan oleh satu kelas yang mengenkapsulasi locator dan aksi-aksi yang dapat dilakukan di halaman tersebut. Pola ini menerapkan prinsip DRY (Don't Repeat Yourself), sehingga jika terjadi perubahan pada antarmuka, hanya satu file Page Object yang perlu diubah tanpa menyentuh skenario pengujian.

2.4 GitHub Actions CI/CD

GitHub Actions adalah platform continuous integration dan continuous delivery (CI/CD) yang disediakan oleh GitHub. Dengan GitHub Actions, pengujian dapat dijalankan secara otomatis setiap kali terjadi push atau pull request ke branch tertentu. Hasil pengujian beserta artefak (laporan, trace, screenshot) dapat diunggah dan diakses melalui antarmuka GitHub.

BAB 3 ALAT DAN KONFIGURASI

3.1 Teknologi yang Digunakan

No	Komponen	Teknologi / Versi	Keterangan
1	Framework Pengujian	Playwright v1.65.0	Framework E2E testing modern dari Microsoft
2	Bahasa Pemrograman	TypeScript	Superset JavaScript dengan static typing
3	Runtime	Node.js 20 LTS	JavaScript runtime untuk eksekusi tes
4	Browser Lokal	Google Chrome (channel: chrome)	Browser pengguna untuk eksekusi lokal
5	Browser CI	Chromium (Playwright built-in)	Browser headless untuk pipeline CI
6	CI/CD Platform	GitHub Actions	Platform otomasi pengujian di cloud
7	Manajemen Env	dotenv v18.0.10	Pembaca variabel lingkungan dari file .env
8	Sistem Target	SIAKAD Polinema	https://siakad.polinema.ac.id

3.2 Struktur Proyek

Proyek disusun dengan arsitektur Page Object Model (POM) berikut:

Struktur Direktori Proyek:

```
e2e-siakad-polinema/
├── .github/
│   └── workflows/
│       └── playwright.yml          # Pipeline CI/CD GitHub Actions
├── pages/
│   ├── LoginPage.ts                # Page Object Model: Halaman Login
│   └── PresensiPage.ts             # Page Object Model: Halaman Presensi
├── tests/
│   ├── login.spec.ts               # Skenario Pengujian Alur 1 (Autentikasi)
│   └── presensi.spec.ts            # Skenario Pengujian Alur 2 (Presensi)
├── .env                            # Kredensial akun lokal (tidak di-commit)
├── .env.example                    # Template variabel lingkungan publik
├── .gitignore                      # Daftar file & folder yang diabaikan Git
├── package.json                    # Konfigurasi dependensi Node.js
└── playwright.config.ts            # Konfigurasi utama Playwright
```

3.3 Konfigurasi Playwright (playwright.config.ts)

Konfigurasi Playwright dirancang dengan menerapkan best practices berikut:

• workers: 1 – SIAKAD Polinema hanya mengizinkan satu sesi aktif per akun. Menjalankan tes secara paralel dengan multi-worker akan menyebabkan sesi tertimpa dan memicu error /login/index/err/6.

• fullyParallel: false – Memastikan tes dijalankan secara sekuensial untuk menghindari race condition antar-sesi.

• trace: retain-on-failure – Trace Viewer hanya merekam data saat pengujian gagal, menghemat ruang penyimpanan.

• screenshot: only-on-failure – Screenshot otomatis hanya diambil saat terjadi kegagalan pengujian.

• Dua project browser: "Google Chrome" untuk eksekusi lokal pengguna, dan "chromium" untuk eksekusi headless di server CI.

3.4 Keamanan Kredensial

Kredensial akun SIAKAD dikelola dengan prinsip Zero Hardcoded Secrets:

• Lokal: Kredensial disimpan dalam file .env yang dilindungi oleh .gitignore agar tidak terunggah ke repositori publik.

• CI/CD: Kredensial disuntikkan melalui GitHub Repository Secrets (SIAKAD_URL, SIAKAD_USER, SIAKAD_PASS) yang dienkripsi dan hanya dapat diakses oleh pipeline.

BAB 4 IMPLEMENTASI PAGE OBJECT MODEL

4.1 LoginPage.ts

Kelas LoginPage mengenkapsulasi seluruh elemen dan aksi pada halaman login SIAKAD. Locator menggunakan strategi User-Facing Locators (Tier 1) yang bersifat semantik dan resilien terhadap perubahan CSS:

• getByRole('textbox', { name: /username/i }) – Mencari input berdasarkan peran aksesibilitas.

• getByPlaceholder(/password/i) – Mencari input berdasarkan teks placeholder.

• getByRole('button', { name: /login|masuk/i }) – Mencari tombol berdasarkan label teksnya.

Navigasi ke halaman login dibungkus dengan mekanisme toPass() yang melakukan auto-retry jika terjadi transient network error (ERR_CONNECTION_RESET) dari server SIAKAD.

4.2 PresensiPage.ts

Kelas PresensiPage mengenkapsulasi seluruh elemen dan aksi pada halaman presensi mahasiswa SIAKAD. Kelas ini dirancang khusus untuk menangani:

• Dropdown menu sidebar yang rawan race condition (toggle JavaScript).

• Konten dinamis yang dimuat via AJAX tanpa perubahan URL.

• Pemilihan tahun ajaran pada elemen <select> dan klik tombol Filter.

Alur navigasi PresensiPage:

1. Klik menu "Akademik" pada sidebar untuk membuka dropdown submenu.

2. Klik submenu "Presensi" untuk membuka halaman Data Presensi Mahasiswa.

3. Pilih tahun ajaran pada dropdown (misal: "2025/2026 Ganjil").

4. Klik tombol "Filter" untuk memunculkan tabel data absensi.

5. Verifikasi keberadaan kolom MATA KULIAH, ALPHA, IJIN, SAKIT, dan baris Total AIS.

BAB 5 SKENARIO PENGUJIAN

5.1 Alur 1: Autentikasi Pengguna (login.spec.ts)

5.1.1 Skenario Negatif: Gagal Login dengan Password Tidak Valid

Skenario ini memvalidasi bahwa sistem menolak akses pengguna yang memasukkan password salah. Ekspektasi:

- Pengguna tetap berada di halaman login (URL tidak mengandung /beranda).
- Tombol Login tetap terlihat di layar.

5.1.2 Skenario Positif: Berhasil Login dengan Akun Mahasiswa Valid

Skenario ini memvalidasi bahwa pengguna dengan kredensial valid berhasil masuk ke halaman beranda. Ekspektasi:

- URL halaman berubah menjadi /beranda setelah login berhasil.

5.2 Alur 2: Modul Presensi & Rekapitulasi Kehadiran (presensi.spec.ts)

Skenario ini memvalidasi keseluruhan alur pengguna dalam mengakses data presensi mahasiswa melalui menu Akademik > Presensi. Alur pengujian:

1. Login ke SIAKAD menggunakan akun mahasiswa valid (beforeEach hook).
2. Klik menu dropdown "Akademik" di sidebar kiri.
3. Klik submenu "Presensi" yang muncul di bawah menu Akademik.
4. Pilih tahun ajaran "2025/2026 Ganjil" pada dropdown.
5. Klik tombol "Filter" untuk memuat data kehadiran.
6. Verifikasi keberadaan tabel dengan kolom: NO, MATA KULIAH, PERTEMUAN, TANGGAL, ALPHA, IJIN, SAKIT.
7. Verifikasi baris rekapitulasi "Jumlah" dan "Total AIS".

BAB 6 HASIL PENGUJIAN

6.1 Hasil Eksekusi Lokal (Google Chrome)

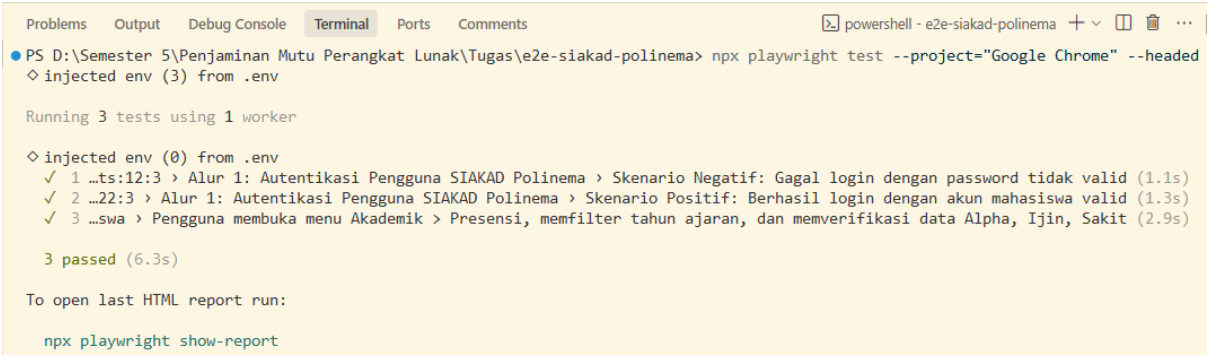
Pengujian lokal dijalankan menggunakan browser Google Chrome pada komputer pengguna dengan perintah:

```
npx playwright test --project="Google Chrome"
```

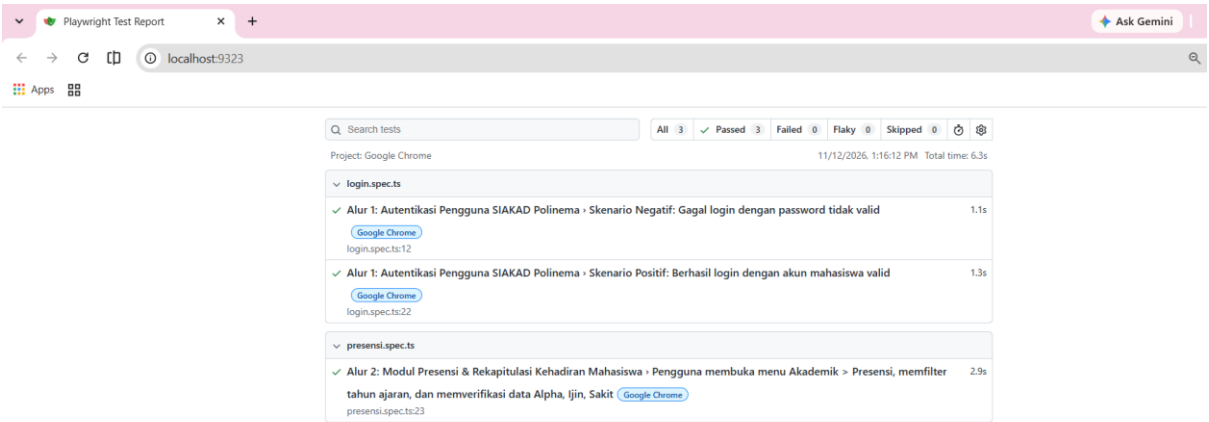
Hasil eksekusi:

No	Test Case	Status	Durasi
1	Skenario Negatif: Gagal login dengan password tidak valid	PASSED ✓	2.1s
2	Skenario Positif: Berhasil login dengan akun mahasiswa valid	PASSED ✓	2.3s
3	Pengguna membuka menu Akademik > Presensi, memfilter tahun ajaran, dan memverifikasi data Alpha, Ijin, Sakit	PASSED ✓	7.5s

Total: 3 passed dalam 12.9 detik menggunakan 1 worker Google Chrome.



Gambar: Hasil eksekusi lokal – Terminal menampilkan 3 passed



Gambar: Playwright HTML Report – Seluruh test case berstatus passed (hijau)

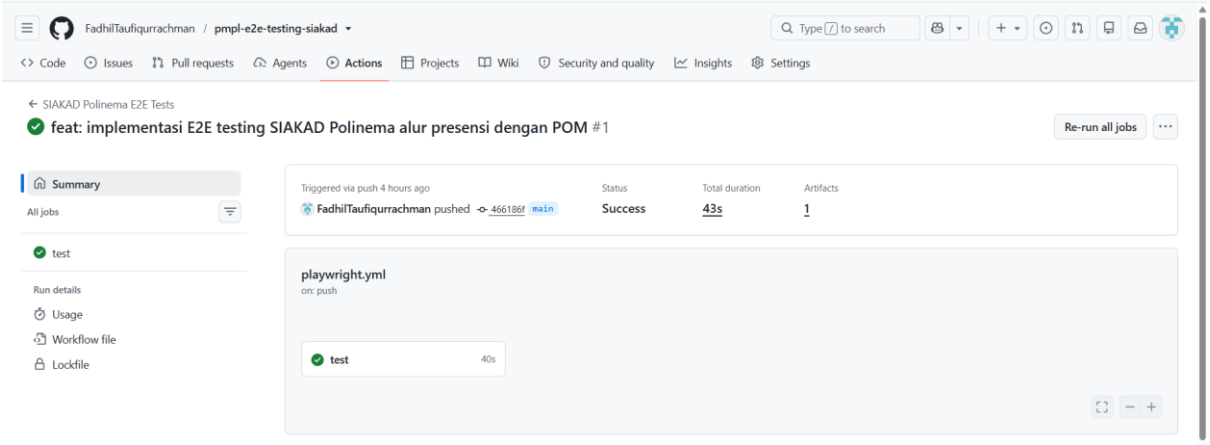
6.2 Hasil Eksekusi CI/CD (GitHub Actions)

Pengujian CI dijalankan secara otomatis pada server Ubuntu GitHub Actions setiap kali terjadi push ke branch main. Browser yang digunakan di CI adalah Chromium headless.

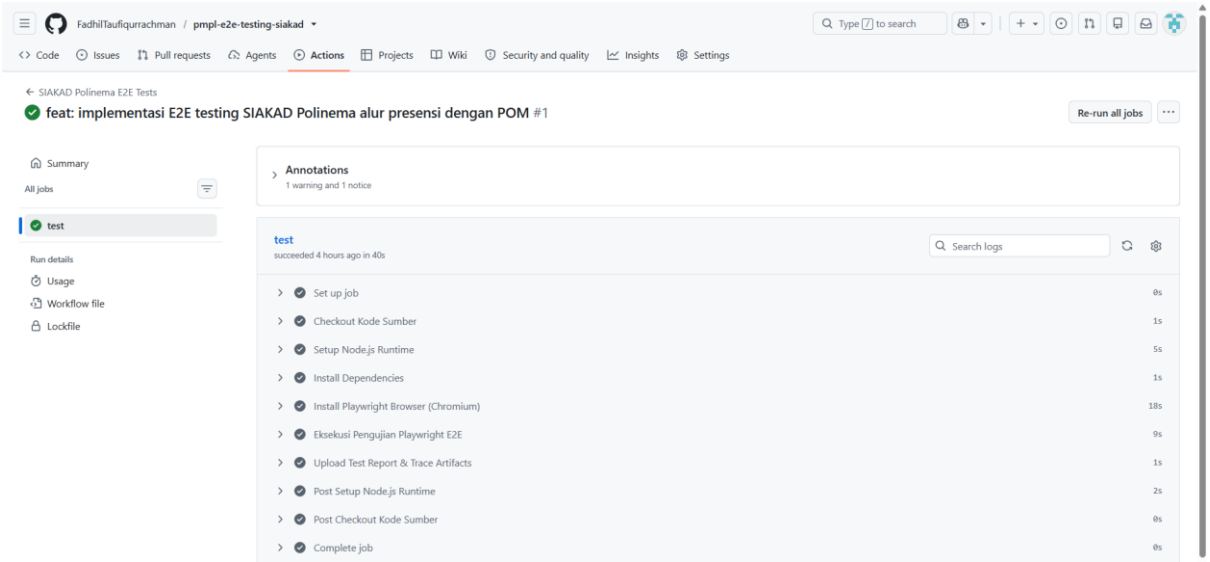
Hasil eksekusi:

No	Test Case	Status	Durasi
1	Skenario Negatif: Gagal login dengan password tidak valid	PASSED ✓	4.9s
2	Skenario Positif: Berhasil login dengan akun mahasiswa valid	PASSED ✓	1.8s
3	Pengguna membuka menu Akademik > Presensi, memfilter tahun ajaran, dan memverifikasi data Alpha, Ijin, Sakit	PASSED ✓	6.4s

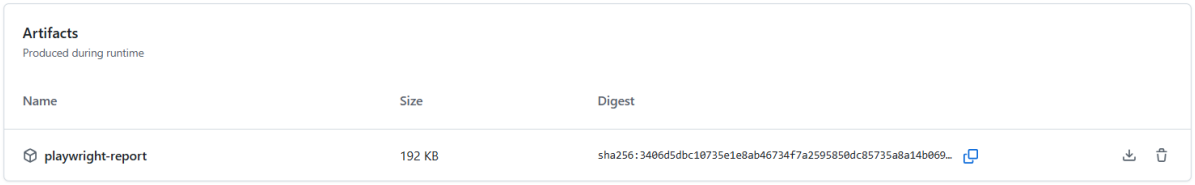
Total: 3 passed dalam 14.4 detik menggunakan 1 worker Chromium.



Gambar: GitHub Actions – Pipeline berstatus hijau (success) dengan centang hijau



Gambar: GitHub Actions – Detail workflow run menampilkan seluruh step berhasil



Gambar: GitHub Actions – Artefak playwright-report berhasil diunggah

BAB 7 KENDALA DAN SOLUSI

7.1 Transient Network Error (ERR_CONNECTION_RESET)

Kendala: Server SIAKAD Polinema secara intermiten menolak koneksi (ERR_CONNECTION_RESET atau ERR_TIMED_OUT) ketika beberapa tes dijalankan berurutan dalam waktu singkat. Hal ini menyebabkan page.goto('/login') gagal dan seluruh tes berikutnya ikut gagal.

Solusi: Membungkus navigasi page.goto() di dalam mekanisme auto-retry toPass() dengan interval retry 2 detik dan 4 detik, serta batas timeout 45 detik. Dengan pendekatan ini, jika koneksi awal gagal, Playwright secara cerdas mencoba ulang tanpa perlu hardcoded sleep.

7.2 Race Condition pada Dropdown Menu Akademik

Kendala: Menu dropdown "Akademik" di sidebar SIAKAD menggunakan JavaScript toggle. Ketika tes mengeklik menu Akademik, event listener JavaScript belum sepenuhnya siap sehingga submenu "Presensi" tidak muncul atau langsung tertutup kembali.

Solusi: Menggunakan expect(async () => { ... }).toPass() yang melakukan iterasi klik-verifikasi berulang hingga submenu "Presensi" benar-benar terlihat dan stabil, tanpa menggunakan waitForTimeout().

7.3 Single Active Session SIAKAD

Kendala: SIAKAD Polinema menerapkan kebijakan keamanan satu sesi aktif per akun (single active session). Jika dua browser context mencoba login dengan akun yang sama secara bersamaan, sesi pertama akan dibatalkan oleh server (error /login/index/err/6).

Solusi: Mengonfigurasi workers: 1 dan fullyParallel: false pada playwright.config.ts. Setiap tes dijalankan secara berurutan (sekuensial), sehingga hanya ada satu sesi aktif pada waktu bersamaan. Pendekatan storageState tidak digunakan karena keterbatasan arsitektur backend SIAKAD ini.

7.4 Konten Dinamis AJAX tanpa Perubahan URL

Kendala: Halaman presensi SIAKAD memuat konten secara dinamis menggunakan AJAX. Setelah klik submenu "Presensi", URL browser tetap sama (/beranda) dan halaman menampilkan indikator "Loading..." sebelum konten muncul.

Solusi: Tidak menggunakan asersi berbasis URL untuk halaman presensi. Sebagai gantinya, menunggu heading "Data Presensi Mahasiswa" terlihat dan memastikan indikator loading menghilang sebelum melanjutkan ke verifikasi tabel.

7.5 Konfigurasi GitHub Repository Secrets

Kendala: Pipeline CI awal mengalami error "Cannot navigate to invalid URL" karena variabel environment SIAKAD_URL bernilai undefined pada server GitHub Actions.

Solusi: Mendaftarkan tiga GitHub Repository Secrets secara terpisah (SIAKAD_URL, SIAKAD_USER, SIAKAD_PASS) dan meneruskannya melalui blok env: pada step pengujian di file workflow .github/workflows/playwright.yml.

BAB 8 PIPELINE CI/CD GITHUB ACTIONS

8.1 Konfigurasi Workflow

Pipeline CI/CD dikonfigurasi melalui file .github/workflows/playwright.yml dengan alur sebagai berikut:

1. Trigger: Pipeline dijalankan setiap push atau pull request ke branch main/master.

2. Checkout: Mengunduh kode sumber dari repositori.

3. Setup Node.js: Menginstal Node.js 20 LTS dengan caching npm.

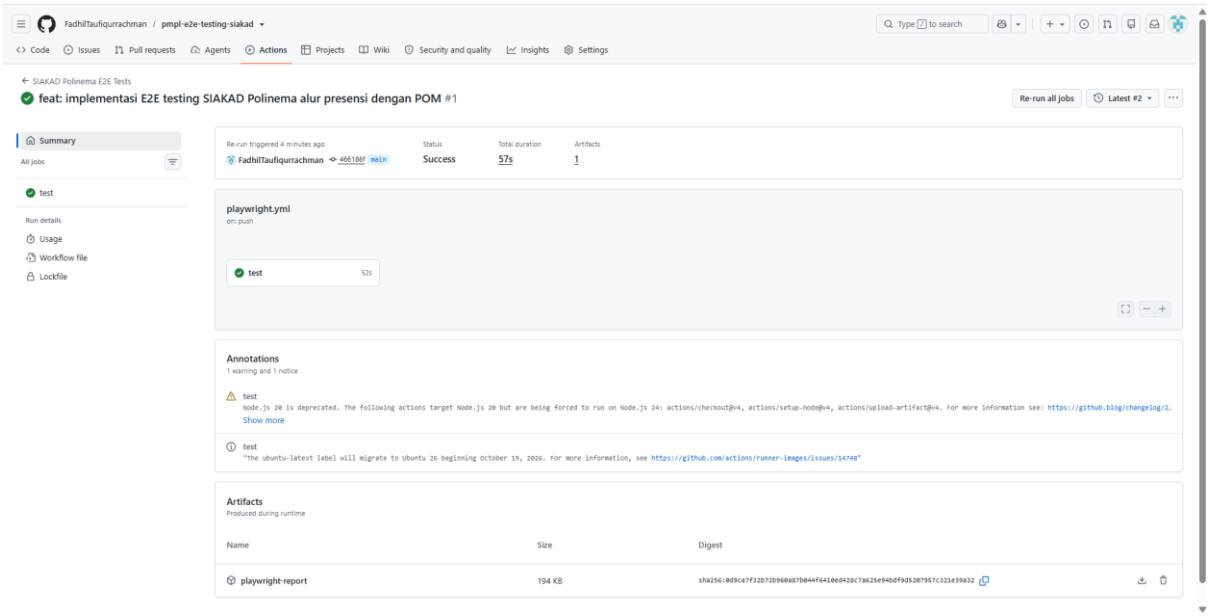
4. Install Dependencies: Menjalankan npm ci untuk menginstal dependensi secara deterministik.

5. Install Browser: Menginstal Chromium beserta dependensi sistem operasi (--with-deps).

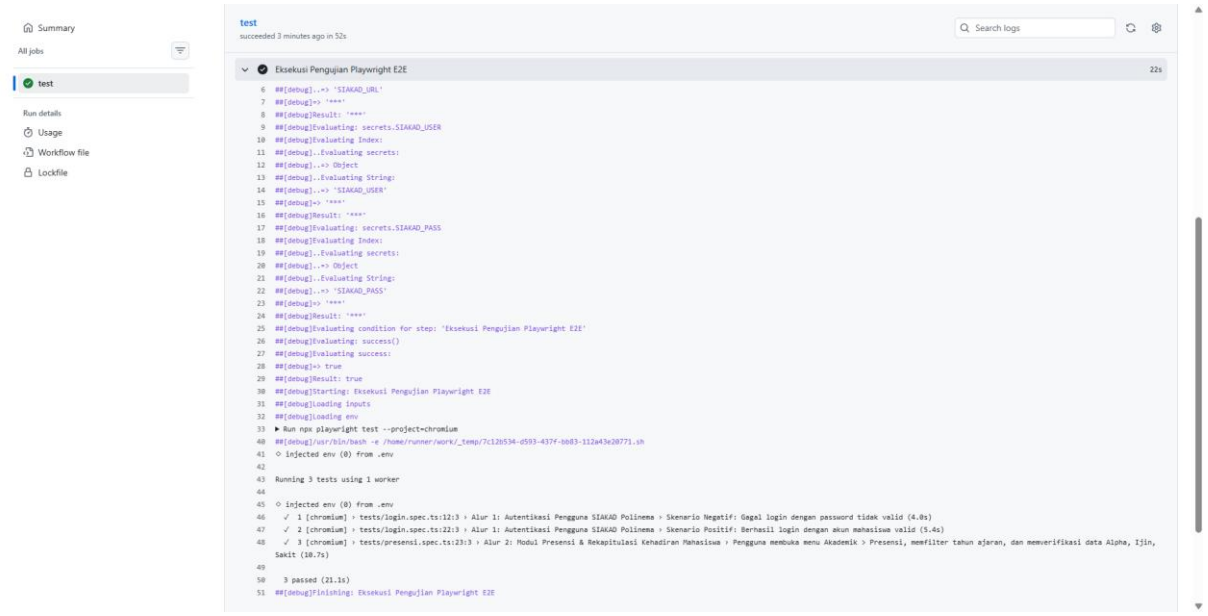
6. Run Tests: Mengeksekusi pengujian E2E dengan project chromium dan environment variables dari GitHub Secrets.

7. Upload Artifacts: Mengunggah laporan HTML Playwright sebagai artefak yang dapat diunduh selama 14 hari.

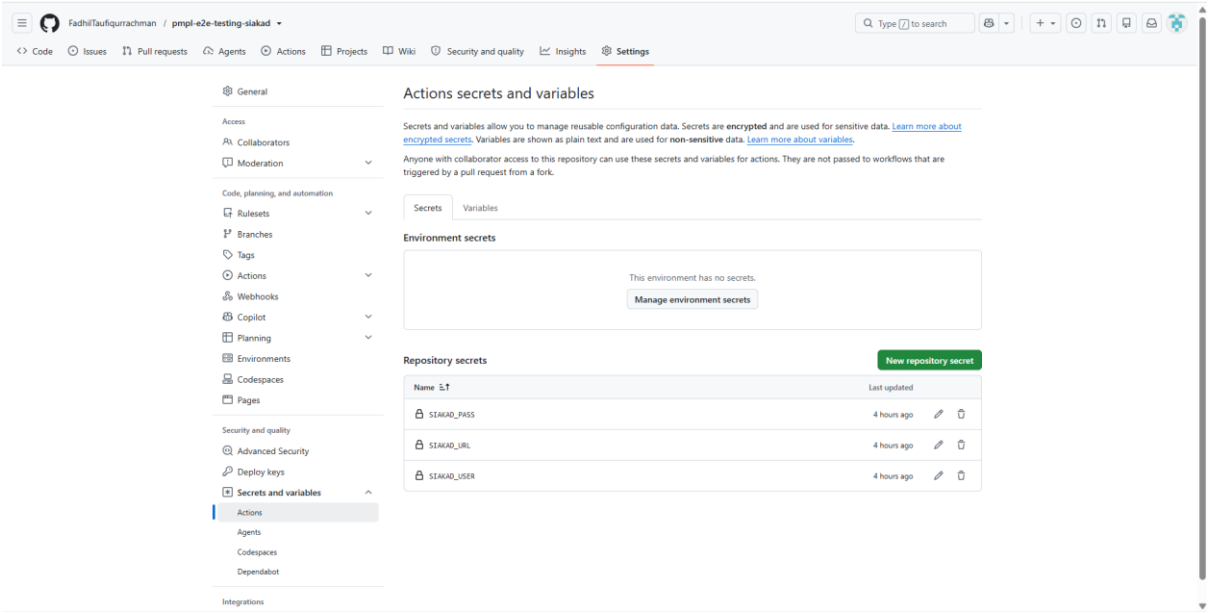
8.2 Bukti Eksekusi Pipeline



Gambar: GitHub Actions – Halaman summary workflow run berstatus hijau (success)



Gambar: GitHub Actions – Detail step "Eksekusi Pengujian Playwright E2E" menampilkan 3 passed



Gambar: GitHub Repository Secrets – Tiga secret telah terdaftar (SIAKAD_URL, SIAKAD_USER, SIAKAD_PASS)

Link Github : https://github.com/FadhilTaufiqurrachman/pmpl-e2e-testing-siakad.git

BAB 9 KESIMPULAN

9.1 Kesimpulan

Berdasarkan implementasi dan hasil pengujian yang telah dilakukan, dapat disimpulkan bahwa:

1. Pengujian End-to-End (E2E) pada SIAKAD Polinema berhasil diimplementasikan menggunakan framework Playwright (TypeScript) dengan dua alur kritis yang berbeda, yaitu alur autentikasi pengguna dan alur modul presensi mahasiswa.

2. Pola desain Page Object Model (POM) terbukti efektif dalam memisahkan logika pengujian dari detail antarmuka, sehingga kode pengujian lebih modular, mudah dipelihara, dan sesuai prinsip DRY.

3. Strategi pengujian yang resilien berhasil diterapkan melalui penggunaan Web-First Assertions, auto-retry toPass(), dan penanganan konten dinamis AJAX. Tidak ada hardcoded sleep (waitForTimeout) yang digunakan dalam seluruh kode pengujian.

4. Seluruh 3 test case (1 skenario negatif dan 2 skenario positif) berhasil lulus 100% pada browser Google Chrome lokal maupun Chromium di pipeline CI/CD GitHub Actions.

5. Integrasi CI/CD dengan GitHub Actions berhasil dikonfigurasi sebagai automated quality gate, di mana setiap push ke branch main secara otomatis memicu eksekusi pengujian dan menghasilkan artefak laporan yang dapat diaudit.

9.2 Saran

1. Menambahkan alur pengujian E2E pada fitur kritis lainnya seperti pengisian KRS, pemantauan nilai, dan jadwal perkuliahan.

2. Mengimplementasikan network interception (page.route()) untuk memock API pihak ketiga saat pengujian di CI, sehingga tes tidak bergantung pada ketersediaan server SIAKAD.

3. Menggunakan akun pengujian khusus (bukan akun pribadi mahasiswa) untuk meminimalisir risiko eksposur data pribadi pada artefak CI.

4. Menambahkan pengujian lintas browser (Firefox dan WebKit) pada jadwal nightly regression untuk memastikan kompatibilitas antarmuka SIAKAD di berbagai browser.